UNIVERSITY OF ENGINEERING AND TECHNOLOGY

Nhóm 7

HỆ THỐNG CƠ SỞ DỮ LIỆU PHÂN TÍCH DỮ LIỆU CHO WEB E-COMMERCE

GROUP ASSIGNMENT REPORT

Subject : Hệ quản trị cơ sở dữ liệu

Hanoi, 2026

Author List

STT	Họ và tên	MSSV	Vai trò
1	Nuyễn Bình Dương	21020001	Nhóm trưởng
2	Nguyễn Văn Huy	21020002	Thành viên
3	Nguyễn Khánh An	21020003	Thành viên
4	Nguyễn Hai Bảy	21020004	Thành viên
5	Nguyễn Minh Đức	21020003	Thành viên
6	Lò Trí Dũng	21020004	Thành viên

Nhóm 7

HỆ THỐNG CƠ SỞ DỮ LIỆU PHÂN TÍCH DỮ LIỆU CHO WEB E-COMMERCE

Abstract

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like "Huardest gefburn"? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like "Huardest gefburn"? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Contents

Tóm tắt **i**

Danh mục hình vẽ **ix**

Danh mục bảng biểu **x**

Mở đầu **xi**

1 Phân tích hệ thống **1**
- 1.1 Mục tiêu của hệ thống 1
- 1.2 Các tác nhân của hệ thống 2
  - 1.2.1 Khách hàng (Buyer/Customer) 2
  - 1.2.2 Người bán (Seller/Seller-Account) 2
  - 1.2.3 Quản trị viên (Administrator) 2
- 1.3 Yêu cầu đối với hệ thống 2
  - 1.3.1 Yêu cầu chức năng 2
  - 1.3.2 Yêu cầu phi chức năng 3

2 Thiết kế hệ thống **4**
- 2.1 Xác định các thực thể 4
- 2.2 Xác định các quan hệ 5
  - 2.2.1 User và Account 5
  - 2.2.2 Các quan hệ khác 5
- 2.3 Biểu đồ thực thể - liên kết 5
- 2.4 Lược đồ quan hệ và mô hình quan hệ 5
  - 2.4.1 Chuẩn hóa dữ liệu 6
  - 2.4.2 Danh sách các bảng chính 6

3 Triển khai cài đặt cơ sở dữ liệu 7
3.1 Account . . . 7
3.1.1 Cấu trúc bảng . . . 7
3.1.2 Index . . . 8
3.1.3 View . . . 8
3.1.4 Trigger . . . 8
3.1.5 Procedure . . . 9
3.1.6 Function . . . 9
3.2 User . . . 9
3.2.1 Table users . . . 9
3.2.1.1 Cấu trúc bảng . . . 9
3.2.1.2 Index . . . 9
3.2.1.3 View . . . 10
3.2.1.4 Trigger . . . 10
3.2.1.5 Procedure . . . 10
3.2.1.6 Function . . . 10
3.2.2 Table addresses . . . 10
3.2.2.1 Cấu trúc bảng . . . 10
3.2.2.2 Index . . . 10
3.2.2.3 View . . . 10
3.2.2.4 Trigger . . . 10
3.2.2.5 Procedure . . . 10
3.2.2.6 Function . . . 10
3.2.3 Table phone_numbers . . . 10
3.2.3.1 Cấu trúc bảng . . . 11
3.2.3.2 Index . . . 11
3.2.3.3 View . . . 11
3.2.3.4 Trigger . . . 11
3.2.3.5 Procedure . . . 11
3.2.3.6 Function . . . 11
3.3 Product . . . 11
3.3.1 Table product . . . 11
3.3.1.1 Cấu trúc bảng . . . 11
3.3.1.2 Index . . . 11
3.3.1.3 View . . . 11

3.3.1.4 Trigger 11
3.3.1.5 Procedure 14
3.3.1.6 Function 14
3.3.2 Table type 14
3.3.2.1 Cấu trúc bảng 14
3.3.2.2 Index 14
3.3.2.3 View 14
3.3.2.4 Trigger 14
3.3.2.5 Procedure 14
3.3.2.6 Function 14
3.3.3 Table type_values 14
3.3.3.1 Cấu trúc bảng 14
3.3.3.2 Index 14
3.3.3.3 View 14
3.3.3.4 Trigger 14
3.3.3.5 Procedure 14
3.3.3.6 Function 14
3.3.4 Table variants 14
3.3.4.1 Cấu trúc bảng 14
3.3.4.2 Index 14
3.3.4.3 View 14
3.3.4.4 Trigger 14
3.3.4.5 Procedure 17
3.3.4.6 Function 19
3.3.5 Table variant_values 21
3.3.5.1 Cấu trúc bảng 21
3.3.5.2 Index 21
3.3.5.3 View 21
3.3.5.4 Trigger 21
3.3.5.5 Procedure 21
3.3.5.6 Function 21
3.3.6 Table product_extra_images 21
3.3.6.1 Cấu trúc bảng 21
3.3.6.2 Index 21
3.3.6.3 View 21

3.3.6.4 Trigger 21
3.3.6.5 Procedure 22
3.3.6.6 Function 22
3.3.7 Table product_reviews 22
3.3.7.1 Cấu trúc bảng 22
3.3.7.2 Index 22
3.3.7.3 View 22
3.3.7.4 Trigger 22
3.3.7.5 Procedure 22
3.3.7.6 Function 24
3.3.8 Table product_daily 25
3.3.8.1 Cấu trúc bảng 25
3.3.8.2 Index 25
3.3.8.3 View 25
3.3.8.4 Trigger 25
3.3.8.5 Procedure 25
3.3.8.6 Function 26
3.4 Category 26
3.4.1 Cấu trúc bảng 27
3.4.2 Index 27
3.4.3 View 27
3.4.3.1 Trigger 27
3.4.3.2 Procedure 27
3.4.3.3 Function 29
3.5 Order 29
3.5.1 Table orders 29
3.5.1.1 Cấu trúc bảng 29
3.5.1.2 Index 30
3.5.1.3 View 30
3.5.1.4 Trigger 31
3.5.1.5 Procedure 32
3.5.1.6 Function 33
3.5.2 Table order_tracking_locations 34
3.5.2.1 Cấu trúc bảng 34
3.5.2.2 Index 35

3.5.2.3 View 35
3.5.2.4 Trigger 35
3.5.2.5 Procedure 36
3.5.2.6 Function 36
3.6 Transaction 36
3.6.1 Cấu trúc bảng 36
3.6.2 Index 36
3.6.3 View 36
3.6.4 Trigger 36
3.6.4.1 Procedure 36
3.6.4.2 Function 36
3.7 Notification 36
3.7.1 Cấu trúc bảng 36
3.7.2 Index 37
3.7.3 View 37
3.7.4 Trigger 38
3.8 Shopping Cart 39
3.8.1 Table shopping_carts 39
3.8.1.1 Cấu trúc bảng 39
3.8.1.2 Index 39
3.8.1.3 Trigger 39
3.8.2 Table shopping_cart_items 40
3.8.2.1 Cấu trúc bảng 40
3.8.2.2 Index 40
3.8.2.3 Trigger 41
3.8.3 View 41
3.9 Refund 42
3.9.1 Table order_refund_requests 42
3.9.1.1 Cấu trúc bảng 42
3.9.1.2 Index 43
3.9.1.3 View 43
3.9.1.4 Trigger 43
3.9.1.5 Procedure 44
3.9.1.6 Function 45

4 Triển khai các chức năng bổ sung cho hệ thống 47

4.1 Chức năng cho guest (chưa đăng nhập) 47
4.1.1 Đăng ký 47
4.1.1.1 Giải thích cơ chế 47
4.1.1.2 Cài đặt vật lý 48
4.1.1.3 Câu lệnh truy vấn 48
4.1.2 Đăng nhập 49
4.1.2.1 Giải thích cơ chế 49
4.1.2.2 Cài đặt vật lý 49
4.1.2.3 Câu lệnh truy vấn 49
4.1.3 Đăng xuất 49
4.1.3.1 Giải thích cơ chế 49
4.1.3.2 Cài đặt vật lý 49
4.1.3.3 Câu lệnh truy vấn 49
4.1.4 Hiển thị danh mục được tìm kiếm nhiều 49
4.1.4.1 Giải thích cơ chế 49
4.1.4.2 Cài đặt vật lý 49
4.1.4.3 Câu lệnh truy vấn 49
4.1.5 Hiển thị sản phẩm bán chạy 49
4.1.5.1 Giải thích cơ chế 49
4.1.5.2 Cài đặt vật lý 49
4.1.5.3 Câu lệnh truy vấn 49
4.1.6 Hiển thị sản phẩm xem nhiều nhất 49
4.1.6.1 Giải thích cơ chế 49
4.1.6.2 Cài đặt vật lý 49
4.1.6.3 Câu lệnh truy vấn 49
4.1.7 Lọc sản phẩm đa tiêu chí 49
4.1.7.1 Giải thích cơ chế 49
4.1.7.2 Cài đặt vật lý 49
4.1.7.3 Câu lệnh truy vấn 49
4.2 Chức năng cho Khách hàng (Người mua) 49
4.2.1 Xem tổng chi tiêu cá nhân 49
4.2.1.1 Giải thích cơ chế 49
4.2.1.2 Cài đặt vật lý 49
4.2.1.3 Câu lệnh truy vấn 49
4.2.2 Hiển thị lịch sử giao dịch 49

4.2.2.1 Giải thích cơ chế 49
4.2.2.2 Cài đặt vật lý 49
4.2.2.3 Câu lệnh truy vấn 49
4.3 Chức năng cho Người bán (Seller) 49
4.3.1 Hiển thị sản phẩm bán chạy của shop 49
4.3.1.1 Giải thích cơ chế 49
4.3.1.2 Cài đặt vật lý 49
4.3.1.3 Câu lệnh truy vấn 49
4.3.2 Hiển thị sản phẩm có tỉ suất mua cao 49
4.3.2.1 Giải thích cơ chế 49
4.3.2.2 Cài đặt vật lý 49
4.3.2.3 Câu lệnh truy vấn 49
4.3.3 Tỉ lệ hủy/ hoàn hàng 49
4.3.3.1 Giải thích cơ chế 49
4.3.3.2 Cài đặt vật lý 49
4.3.3.3 Câu lệnh truy vấn 49
4.3.4 Gợi ý nhập kho 49
4.3.4.1 Giải thích cơ chế 49
4.3.4.2 Cài đặt vật lý 49
4.3.4.3 Câu lệnh truy vấn 49
4.3.5 Dự báo cạn kho 49
4.3.5.1 Giải thích cơ chế 49
4.3.5.2 Cài đặt vật lý 49
4.3.5.3 Câu lệnh truy vấn 49
4.3.6 Tăng trưởng của sản phẩm 49
4.3.6.1 Giải thích cơ chế 49
4.3.6.2 Cài đặt vật lý 49
4.3.6.3 Câu lệnh truy vấn 49
4.3.7 Biểu đồ doanh thu theo ngày 49
4.3.7.1 Giải thích cơ chế 49
4.3.7.2 Cài đặt vật lý 49
4.3.7.3 Câu lệnh truy vấn 49
4.3.8 Hiển thị danh mục bán chạy của shop 49
4.3.8.1 Giải thích cơ chế 49
4.3.8.2 Cài đặt vật lý 49

4.3.8.3 Câu lệnh truy vấn 49
4.3.9 Hiển thị danh mục được xem nhiều 49
4.3.9.1 Giải thích cơ chế 49
4.3.9.2 Cài đặt vật lý 49
4.3.9.3 Câu lệnh truy vấn 49
4.3.10 Hiển thị sản phẩm được xem nhiều 49
4.3.10.1 Giải thích cơ chế 49
4.3.10.2 Cài đặt vật lý 49
4.3.10.3 Câu lệnh truy vấn 49
4.3.11 Hiển thị phân bố doanh thu của sub-category 49
4.3.11.1 Giải thích cơ chế 49
4.3.11.2 Cài đặt vật lý 49
4.3.11.3 Câu lệnh truy vấn 49
4.4 Chức năng cho Admin (Quản trị viên) 49
4.4.1 Hiển thị người dùng có hành vi xấu 49
4.4.1.1 Giải thích cơ chế 49
4.4.1.2 Cài đặt vật lý 49
4.4.1.3 Câu lệnh truy vấn 49
4.4.2 4.5.2 Hiển thị doanh thu của toàn sàn 49
4.4.2.1 Giải thích cơ chế 49
4.4.2.2 Cài đặt vật lý 49
4.4.2.3 Câu lệnh truy vấn 49

References **50**

Appendices **51**

A Conclusion **51**
A.1 Điểm mạnh 51
A.2 Điểm yếu 51
A.3 Kết luận Cuối 51

B Khuyến nghị và Hướng phát triển **52**

List of Figures

2.3.1 Biểu đồ thực thể - liên kết của hệ thống 5

List of Tables

Mở đầu

Lý do chọn đề tài

Trong bối cảnh thương mại điện tử phát triển nhanh chóng, việc quản lý dữ liệu hiệu quả trở thành yêu cầu cấp thiết. Một hệ thống cơ sở dữ liệu tốt không chỉ lưu trữ thông tin mà còn hỗ trợ các quyết định kinh doanh quan trọng.

Nhóm chúng tôi chọn đề tài này vì:

- Tầm quan trọng của cơ sở dữ liệu trong e-commerce
- Cơ hội áp dụng kiến thức về thiết kế và tối ưu hóa
- Giá trị thực tiễn cao cho các ứng dụng thương mại điện tử

Đóng góp của đề tài

Đề tài này cung cấp:

- **Thiết kế toàn diện:** Mô hình dữ liệu hoàn chỉnh cho nền tảng e-commerce
- **Triển khai thực tế:** Các schema, view, trigger và procedure tối ưu

- **Phân tích dữ liệu:** Hỗ trợ các tính năng phân tích cho ba nhóm người dùng chính
- **Hiệu suất cao:** Sử dụng index và các kỹ thuật tối ưu hóa

Hiểu rõ yêu cầu là bước đầu tiên của bất kỳ dự án nào.

Phân tích hệ thống

Chapter 1

Phân tích hệ thống

PHÂN TÍCH HỆ THỐNG là giai đoạn quan trọng giúp xác định rõ các yêu cầu, mục tiêu và các yếu tố tác động của hệ thống e-commerce. Phần này trình bày chi tiết phân tích các khía cạnh của hệ thống cơ sở dữ liệu.

1.1 Mục tiêu của hệ thống

Mục tiêu chính của hệ thống cơ sở dữ liệu e-commerce bao gồm:

- Lưu trữ hiệu quả thông tin sản phẩm, người dùng và đơn hàng
- Hỗ trợ giao dịch an toàn và tin cậy
- Cung cấp hiệu suất truy vấn cao cho các hoạt động thời gian thực
- Đảm bảo tính toàn vẹn và nhất quán dữ liệu
- Hỗ trợ phân tích dữ liệu chi tiết cho ba nhóm người dùng: Khách hàng, Người bán và Quản trị viên
- Cung cấp cơ sở cho các chức năng báo cáo và dự báo

1.2 Các tác nhân của hệ thống

Hệ thống phục vụ ba nhóm tác nhân chính:

1.2.1 Khách hàng (Buyer/Customer)

- Người dùng chưa đăng nhập (Guest)
- Người dùng đã đăng nhập với vai trò mua hàng

1.2.2 Người bán (Seller/Seller-Account)

- Người bán hàng (Shop Owner)
- Quản lý cửa hàng (Shop Manager)

1.2.3 Quản trị viên (Administrator)

- Quản lý người dùng
- Quản lý giao dịch
- Phân tích dữ liệu toàn hệ thống

1.3 Yêu cầu đối với hệ thống

1.3.1 Yêu cầu chức năng

Hệ thống phải hỗ trợ:

- Quản lý tài khoản (Account Management)
- Quản lý người dùng (User Profile Management)
- Quản lý sản phẩm (Product Management)
- Quản lý danh mục (Category Management)

- Quản lý đơn hàng (Order Management)
- Quản lý giao dịch (Transaction Management)
- Báo cáo và phân tích dữ liệu

1.3.2 Yêu cầu phi chức năng

Hệ thống phải đạt được:

- **Hiệu suất:** Phản hồi nhanh cho các truy vấn thường gặp
- **Tính mở rộng:** Xử lý tăng thêm dữ liệu và lưu lượng người dùng
- **Tính toàn vẹn:** Đảm bảo dữ liệu luôn nhất quán và chính xác
- **Bảo mật:** Bảo vệ dữ liệu nhạy cảm của người dùng
- **Độ tin cậy:** Giảm thiểu thời gian chết hệ thống
- **Khả năng bảo trì:** Dễ dàng sửa chữa, mở rộng và nâng cấp

Chapter 2

Thiết kế hệ thống

Thiết kế hệ thống là quá trình chuyển đổi các yêu cầu phân tích thành mô hình dữ liệu cụ thể. Phần này trình bày chi tiết quá trình thiết kế từ xác định thực thể đến xây dựng lược đồ quan hệ.

2.1 Xác định các thực thể

Các thực thể chính của hệ thống e-commerce bao gồm:

- **Account** - Tài khoản hệ thống
- **User** - Thông tin người dùng
- **Product** - Sản phẩm bán trên nền tảng
- **Category** - Danh mục sản phẩm
- **Order** - Đơn hàng
- **Transaction** - Giao dịch thanh toán

Mỗi thực thể có các thuộc tính riêng biệt xác định đặc trưng và thông tin cần lưu trữ. Các thuộc tính này được chọn lựa dựa trên yêu cầu chức năng của hệ thống.

2.2 Xác định các quan hệ

Các mối quan hệ giữa các thực thể bao gồm:

2.2.1 User và Account

- Mối quan hệ 1:1 - Một User đối ứng với một Account

2.2.2 Các quan hệ khác

(Các quan hệ khác như Product-Category, User-Order, Order-Product, Order-Transaction sẽ được chi tiết trong chương triển khai)

2.3 Biểu đồ thực thể - liên kết

Biểu đồ Entity-Relationship (ERD) biểu diễn trực quan các thực thể, thuộc tính và mối quan hệ của hệ thống. Biểu đồ này giúp hiểu rõ cấu trúc dữ liệu trước khi triển khai.

ĐÂY LÀ MÔ HÌNH ERD

Figure 2.3.1: Biểu đồ thực thể - liên kết của hệ thống

Ghi chú: ERD chi tiết có thể được xem tại: Link ERD chi tiết

2.4 Lược đồ quan hệ và mô hình quan hệ

Lược đồ quan hệ (Relational Schema) chuyển đổi mô hình ER thành các bảng trong CSDL quan hệ. Các bảng được thiết kế tuân theo các quy tắc chuẩn hóa (Normalization) để:

- Giảm dư thừa dữ liệu
- Đảm bảo tính nhất quán
- Tối ưu hóa hiệu suất truy vấn
- Dễ bảo trì và nâng cấp

2.4.1 Chuẩn hóa dữ liệu

Các bảng được chuẩn hóa từ dạng bình thường 1 (1NF) đến dạng bình thường 3 (3NF) để đảm bảo chất lượng lược đồ:

- **1NF**: Loại bỏ các nhóm lặp lại
- **2NF**: Loại bỏ các phụ thuộc hàm từng phần
- **3NF**: Loại bỏ các phụ thuộc hàm bắc cầu

2.4.2 Danh sách các bảng chính

Các bảng chính của hệ thống bao gồm: accounts, users, user_addresses, user_phone_numbers products, product_types, product_variants, categories, orders, order_items, transactions. Chi tiết cấu trúc của các bảng này sẽ được trình bày trong chương 3 (Triển khai cài đặt cơ sở dữ liệu).

Chapter 3

Triển khai cài đặt cơ sở dữ liệu

Triển khai cài đặt cơ sở dữ liệu là giai đoạn chuyển đổi lược đồ thiết kế thành các bảng, index, view và trigger thực tế. Phần này trình bày chi tiết cấu trúc từng bảng, các ràng buộc, index tối ưu hóa, view để lấy dữ liệu và trigger tự động.

3.1 Account

3.1.1 Cấu trúc bảng

Bảng `accounts` lưu trữ thông tin tài khoản hệ thống.

```
CREATE TABLE accounts (
    account_id INT PRIMARY KEY AUTO_INCREMENT,
    email VARCHAR(255) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    role ENUM('buyer', 'seller', 'admin') NOT NULL,
    status ENUM('active', 'suspended', 'deleted') DEFAULT 'active',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
        ON UPDATE CURRENT_TIMESTAMP
);
```

3.1.2 Index

```
CREATE INDEX idx_accounts_email ON accounts(email);
CREATE INDEX idx_accounts_role ON accounts(role);
CREATE INDEX idx_accounts_status ON accounts(status);
```

3.1.3 View

```
CREATE VIEW vw_active_accounts AS
SELECT account_id, email, role, created_at
FROM accounts
WHERE status = 'active';
```

3.1.4 Trigger

```
DELIMITER $$
CREATE TRIGGER trg_account_audit
AFTER UPDATE ON accounts
FOR EACH ROW
BEGIN
    INSERT INTO account_audit_log
    (account_id, old_status, new_status, changed_at)
    VALUES (NEW.account_id, OLD.status, NEW.status, NOW());
END$$
DELIMITER ;
```

3.1.5 Procedure

3.1.6 Function

3.2 User

3.2.1 Table users

Bảng `users` lưu trữ thông tin chi tiết của người dùng.

3.2.1.1 Cấu trúc bảng

```
CREATE TABLE users (
    user_id INT PRIMARY KEY AUTO_INCREMENT,
    account_id INT NOT NULL UNIQUE,
    first_name VARCHAR(100) NOT NULL,
    last_name VARCHAR(100) NOT NULL,
    date_of_birth DATE,
    gender ENUM('male', 'female', 'other'),
    avatar_url VARCHAR(500),
    bio TEXT,
    total_spending DECIMAL(15, 2) DEFAULT 0,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (account_id) REFERENCES accounts(account_id)
        ON DELETE CASCADE
);
```

3.2.1.2 Index

```
CREATE INDEX idx_users_account_id ON users(account_id);
CREATE INDEX idx_users_full_name ON users(last_name, first_name);
CREATE INDEX idx_users_total_spending ON users(total_spending DESC);
```

3.2.1.3 View

```
CREATE VIEW vw_user_profile AS
SELECT
    u.user_id, u.account_id,
    CONCAT(u.first_name, ' ', u.last_name) as full_name,
    a.email, u.total_spending, u.created_at
FROM users u
JOIN accounts a ON u.account_id = a.account_id;
```

3.2.1.4 Trigger

Không có trigger riêng cho bảng users.

3.2.1.5 Procedure

3.2.1.6 Function

3.2.2 Table addresses

Bảng `user_addresses` lưu trữ các địa chỉ của người dùng.

3.2.2.1 Cấu trúc bảng

3.2.2.2 Index

3.2.2.3 View

3.2.2.4 Trigger

3.2.2.5 Procedure

3.2.2.6 Function

3.2.3 Table phone_numbers

Bảng `user_phone_numbers` lưu trữ các số điện thoại của người dùng.

3.2.3.1 Cấu trúc bảng

3.2.3.2 Index

3.2.3.3 View

3.2.3.4 Trigger

3.2.3.5 Procedure

3.2.3.6 Function

3.3 Product

3.3.1 Table product

3.3.1.1 Cấu trúc bảng

3.3.1.2 Index

3.3.1.3 View

3.3.1.4 Trigger

trigger trg_after_insert_variant:

```
CREATE TRIGGER trg_after_insert_variant
AFTER INSERT ON variants
FOR EACH ROW
BEGIN
    UPDATE products
    SET total_quantity = total_quantity + NEW.quantity
    WHERE id = NEW.id;
END;
```

Mục đích: tăng total_quantity trong products khi thêm sản phẩm mới (variant).

trigger trg_after_insert_order_item_sold:

```
CREATE TRIGGER trg_after_insert_order_item_sold
AFTER INSERT ON order_items
FOR EACH ROW
BEGIN
    UPDATE products p
    JOIN variants v ON p.id = v.product_id
    SET p.total_quantity_sold = p.total_quantity_sold + NEW.quantity
    WHERE v.id = NEW.variant_id;
END;
```

Mục đích: cập nhật total_quantity_sold khi bán sản phẩm.

trigger trg_after_delete_order_item_sold:

```
CREATE TRIGGER trg_after_delete_order_item_sold
AFTER DELETE ON order_items
FOR EACH ROW
BEGIN
    UPDATE products p
    JOIN variants v ON p.id = v.product_id
    SET p.total_quantity_sold = p.total_quantity_sold - OLD.quantity
    WHERE v.id = OLD.variant_id;
END;
```

Mục đích: cập nhật total_quantity_sold khi hoàn hàng.

trigger trg_before_insert_product_price:

```
CREATE TRIGGER trg_before_insert_product_price
BEFORE INSERT ON products
FOR EACH ROW
BEGIN
    SET NEW.final_price = NEW.price * (1 - NEW.discount_percentage / 100);
END;
```

Mục đích: khi có discount_percentage thì tự động tính và cập nhật giá sau giảm (final_price).

trigger trg_soft_delete_product:

```
CREATE TRIGGER trg_soft_delete_product
AFTER UPDATE ON products
FOR EACH ROW
BEGIN
    IF NEW.deleted = 1 AND OLD.deleted = 0 THEN
        UPDATE variants
        SET deleted = 1
        WHERE product_id = NEW.id;
    END IF;
END;
```

Mục đích: khi product bị xóa mềm thì các variants liên quan cũng được xóa mềm.

3.3.1.5 Procedure

3.3.1.6 Function

3.3.2 Table type

3.3.2.1 Cấu trúc bảng

3.3.2.2 Index

3.3.2.3 View

3.3.2.4 Trigger

3.3.2.5 Procedure

3.3.2.6 Function

3.3.3 Table type_values

3.3.3.1 Cấu trúc bảng

3.3.3.2 Index

3.3.3.3 View

3.3.3.4 Trigger

3.3.3.5 Procedure

3.3.3.6 Function

3.3.4 Table variants

3.3.4.1 Cấu trúc bảng

3.3.4.2 Index

3.3.4.3 View

3.3.4.4 Trigger

trigger trg_soft_delete_variant:

```
CREATE TRIGGER trg_soft_delete_variant
AFTER UPDATE ON variants
FOR EACH ROW
BEGIN
    IF NEW.deleted = 1 AND OLD.deleted = 0 THEN
        UPDATE variant_values
        SET deleted = 1
        WHERE variant_id = NEW.id;
    END IF;
END;
```

Mục đích: khi variant bị xóa mềm thì các variant_values liên quan cũng bị xóa mềm, không cần lan ngược lên product.

trigger trg_variant_insert_quantity:

```
CREATE TRIGGER trg_variant_insert_quantity
AFTER INSERT ON variants
FOR EACH ROW
BEGIN
    UPDATE products
    SET total_quantity = total_quantity + NEW.quantity
    WHERE id = NEW.product_id;
END;
```

Mục đích: khi thêm variant mới thì cập nhật lại total_quantity của product tương ứng.

trigger trg_order_item_insert:

```
CREATE TRIGGER trg_order_item_insert
AFTER INSERT ON order_items
FOR EACH ROW
BEGIN
    UPDATE variants
    SET quantity = quantity - NEW.quantity,
        quantity_sold = quantity_sold + NEW.quantity
```

```
    WHERE id = NEW.variant_id;
END;
```

Mục đích: cập nhật quantity và quantity_sold của variant sau khi thêm đơn hàng.

trigger trg_order_item_delete:

```
CREATE TRIGGER trg_order_item_delete
AFTER DELETE ON order_items
FOR EACH ROW
BEGIN
    UPDATE variants
    SET quantity = quantity + OLD.quantity,
        quantity_sold = quantity_sold - OLD.quantity
    WHERE id = OLD.variant_id;
END;
```

Mục đích: cập nhật quantity và quantity_sold của variant sau khi hủy/xóa đơn hàng.

trigger trg_variant_update_delta:

```
CREATE TRIGGER trg_variant_update_delta
AFTER UPDATE ON variants
FOR EACH ROW
BEGIN
    DECLARE a INT;
    DECLARE b INT;

    SET a = NEW.quantity - OLD.quantity;
    SET b = NEW.quantity_sold - OLD.quantity_sold;

    UPDATE products
    SET total_quantity = total_quantity + a,
        total_quantity_sold = total_quantity_sold + b
    WHERE id = NEW.product_id;
END;
```

Mục đích: cập nhật total_quantity và total_quantity_sold của product khi dữ liệu variants thay đổi.

trigger trg_variant_insert:

```
CREATE TRIGGER trg_variant_insert
AFTER INSERT ON variants
FOR EACH ROW
BEGIN
    UPDATE products
    SET total_quantity = total_quantity + NEW.quantity,
        total_quantity_sold = total_quantity_sold + NEW.quantity_sold
    WHERE id = NEW.product_id;
END;
```

Mục đích: sau khi thêm variant vào bảng variants thì cập nhật lại total_quantity và total_quantity_sold ở bảng products.

trigger trg_variant_delete:

```
CREATE TRIGGER trg_variant_delete
AFTER DELETE ON variants
FOR EACH ROW
BEGIN
    UPDATE products
    SET total_quantity = total_quantity - OLD.quantity,
        total_quantity_sold = total_quantity_sold - OLD.quantity_sold
    WHERE id = OLD.product_id;
END;
```

Mục đích: cập nhật lại total_quantity và total_quantity_sold của product sau khi xóa một variant.

3.3.4.5 Procedure

procedure sp_get_products_by_category:

```
DELIMITER $$
```

```
CREATE PROCEDURE sp_get_products_by_category(IN p_category_id INT)
BEGIN
    SELECT *
    FROM products
    WHERE category_id = p_category_id
      AND deleted = 0;
END$$

DELIMITER ;
```

Mục đích: lấy danh sách sản phẩm theo category (chỉ lấy sản phẩm chưa bị xóa mềm).

procedure sp_create_order:

```
DELIMITER $$

CREATE PROCEDURE sp_create_order(
    IN p_user_id INT,
    IN p_variant_id INT,
    IN p_quantity INT
)
BEGIN
    DECLARE oid INT;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
    END;

    START TRANSACTION;

    INSERT INTO orders(user_id)
    VALUES (p_user_id);

    SET oid = LAST_INSERT_ID();
```

```
    INSERT INTO order_items(order_id, variant_id, quantity)
    VALUES (oid, p_variant_id, p_quantity);

    COMMIT;
END$$

DELIMITER ;
```

Mục đích: tạo đơn hàng với transaction, đảm bảo nếu có lỗi thì rollback toàn bộ.

3.3.4.6 Function

function fn_final_price:

```
DELIMITER $$

CREATE FUNCTION fn_final_price(
    p_price DECIMAL(10,2),
    p_discount INT
)
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
    RETURN p_price * (1 - p_discount / 100);
END$$

DELIMITER ;
```

Mục đích: tính giá cuối cùng của sản phẩm sau khi áp dụng phần trăm giảm giá.

function fn_total_stock:

```
DELIMITER $$
```

```
CREATE FUNCTION fn_total_stock(p_product_id INT)
RETURNS INT
DETERMINISTIC
BEGIN
    RETURN (
        SELECT IFNULL(SUM(quantity), 0)
        FROM variants
        WHERE product_id = p_product_id
          AND deleted = 0
    );
END$$

DELIMITER ;
```

Mục đích: tính tổng số lượng tồn kho của một product từ các variants (chỉ tính các variant chưa bị xóa mềm).

function fn_hot_variant_by_product:

```
DELIMITER $$

CREATE FUNCTION fn_hot_variant_by_product(p_product_id INT)
RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE v_id INT;

    SELECT v.id
    INTO v_id
    FROM order_items oi
    JOIN variants v ON oi.variant_id = v.id
    WHERE v.product_id = p_product_id
      AND v.deleted = 0
    GROUP BY v.id
    ORDER BY SUM(oi.quantity) DESC
    LIMIT 1;
```

```
    RETURN v_id;
END$$

DELIMITER ;
```

Mục đích: trả về id của variant bán chạy nhất (hot) trong các variants của một product.

3.3.5 Table variant_values

3.3.5.1 Cấu trúc bảng

3.3.5.2 Index

3.3.5.3 View

3.3.5.4 Trigger

3.3.5.5 Procedure

3.3.5.6 Function

3.3.6 Table product_extra_images

3.3.6.1 Cấu trúc bảng

3.3.6.2 Index

3.3.6.3 View

3.3.6.4 Trigger

trigger trg_soft_delete_product_images:

```
CREATE TRIGGER trg_soft_delete_product_images
AFTER UPDATE ON products
FOR EACH ROW
BEGIN
```

```
    IF NEW.deleted = 1 THEN
        UPDATE product_extra_images
        SET deleted = 1
        WHERE product_id = NEW.id;
    END IF;
END;
```

Mục đích: khi product bị xóa mềm thì các ảnh (product_extra_images) liên quan cũng được xóa mềm.

3.3.6.5 Procedure

3.3.6.6 Function

3.3.7 Table product_reviews

3.3.7.1 Cấu trúc bảng

3.3.7.2 Index

3.3.7.3 View

3.3.7.4 Trigger

3.3.7.5 Procedure

procedure sp_get_reviews:

```
DELIMITER $$

CREATE PROCEDURE sp_get_reviews(IN p_product_id INT)
BEGIN
    SELECT *
    FROM product_reviews
    WHERE product_id = p_product_id
      AND deleted = 0
    ORDER BY created_at DESC;
```

```
END$$

DELIMITER ;
```

Mục đích: lấy danh sách review của một sản phẩm, sắp xếp theo thời gian mới nhất và loại bỏ các review đã bị xóa mềm.

procedure sp_get_products_with_rating:

```
DELIMITER $$

CREATE PROCEDURE sp_get_products_with_rating()
BEGIN
    SELECT p.*,
           fn_avg_rating(p.id) AS avg_rating,
           fn_count_review(p.id) AS total_reviews
    FROM products p
    WHERE p.deleted = 0;
END$$

DELIMITER ;
```

Mục đích: lấy danh sách sản phẩm kèm điểm đánh giá trung bình và tổng số lượt review.

procedure sp_soft_delete_review:

```
DELIMITER $$

CREATE PROCEDURE sp_soft_delete_review(IN p_review_id INT)
BEGIN
    UPDATE product_reviews
    SET deleted = 1
    WHERE id = p_review_id;
END$$

DELIMITER ;
```

Mục đích: thực hiện xóa mềm một product_review theo id.

3.3.7.6 Function

function fn_avg_rating:

```
DELIMITER $$

CREATE FUNCTION fn_avg_rating(p_product_id INT)
RETURNS DECIMAL(3,2)
DETERMINISTIC
BEGIN
    RETURN (
        SELECT IFNULL(AVG(rating), 0)
        FROM product_reviews
        WHERE product_id = p_product_id
          AND deleted = 0
    );
END$$

DELIMITER ;
```

Mục đích: tính điểm đánh giá trung bình (rating) của một product từ các review chưa bị xóa mềm.

function fn_count_review:

```
DELIMITER $$

CREATE FUNCTION fn_count_review(p_product_id INT)
RETURNS INT
DETERMINISTIC
BEGIN
RETURN (
SELECT COUNT(*)
FROM product_reviews
```

```
WHERE product_id = p_product_id
AND deleted = 0
);
END$$

DELIMITER ;
```

Mục đích: Đếm số lượng review của một product chưa bị xóa mềm.

3.3.8 Table product_daily

3.3.8.1 Cấu trúc bảng

3.3.8.2 Index

3.3.8.3 View

3.3.8.4 Trigger

3.3.8.5 Procedure

procedure sp_update_product_daily:

```
DELIMITER $$

CREATE PROCEDURE sp_update_product_daily(IN p_date DATE)
BEGIN
    DECLARE v_revenue DECIMAL(12,2);
    DECLARE v_orders INT;

    -- Tổng doanh thu trong ngày
    SELECT IFNULL(SUM(oi.quantity * v.price), 0)
    INTO v_revenue
    FROM orders o
    JOIN order_items oi ON o.id = oi.order_id
    JOIN variants v ON oi.variant_id = v.id
    WHERE DATE(o.created_at) = p_date;
```

```
    -- Tổng số đơn trong ngày
    SELECT COUNT(*)
    INTO v_orders
    FROM orders
    WHERE DATE(created_at) = p_date;

    -- Cập nhật bảng product_daily
    UPDATE product_daily
    SET total_revenue = v_revenue,
        total_orders = v_orders
    WHERE date = p_date;
END$$

DELIMITER ;
```

Mục đích: tính tổng doanh thu và số lượng đơn hàng trong một ngày và cập nhật vào bảng product_daily.

event ev_update_daily:

```
CREATE EVENT ev_update_daily
ON SCHEDULE EVERY 1 DAY
STARTS CURRENT_DATE + INTERVAL 1 DAY
DO
CALL sp_update_product_daily(CURDATE());
```

Mục đích: tự động chạy procedure mỗi ngày để cập nhật doanh thu và số đơn vào cuối ngày.

3.3.8.6 Function

3.4 Category

Bảng `categories` lưu trữ các danh mục sản phẩm.

3.4.1 Cấu trúc bảng

3.4.2 Index

3.4.3 View

3.4.3.1 Trigger

trigger trg_soft_delete_category:

```
DELIMITER $$

CREATE TRIGGER trg_soft_delete_category
AFTER UPDATE ON categories
FOR EACH ROW
BEGIN
    IF NEW.deleted = 1 AND OLD.deleted = 0 THEN
        UPDATE products
        SET deleted = 1
        WHERE category_id = NEW.id;
    END IF;
END$$

DELIMITER ;
```

Mục đích: khi category bị xóa mềm thì các product thuộc category đó cũng được xóa mềm.

3.4.3.2 Procedure

procedure sp_top_products_by_day_category:

```
DELIMITER $$

CREATE PROCEDURE sp_top_products_by_day_category(
    IN p_date DATE,
```

```
    IN p_category_id INT
)
BEGIN
    SELECT p.id, p.name, SUM(oi.quantity) AS total_sold
    FROM order_items oi
    JOIN orders o ON oi.order_id = o.id
    JOIN variants v ON oi.variant_id = v.id
    JOIN products p ON v.product_id = p.id
    WHERE DATE(o.created_at) = p_date
      AND p.category_id = p_category_id
      AND p.deleted = 0
    GROUP BY p.id
    ORDER BY total_sold DESC
    LIMIT 10;
END$$

DELIMITER ;
```

Mục đích: lấy 10 sản phẩm bán chạy nhất trong một ngày theo từng category.

procedure sp_get_products_by_category:

```
DELIMITER $$

CREATE PROCEDURE sp_get_products_by_category(IN p_category_id INT)
BEGIN
    SELECT *
    FROM products
    WHERE category_id = p_category_id
      AND deleted = 0;
END$$

DELIMITER ;
```

Mục đích: lấy danh sách sản phẩm theo category (chỉ bao gồm sản phẩm chưa bị xóa mềm).

3.4.3.3 Function

function fn_count_products_by_category:

```
DELIMITER $$

CREATE FUNCTION fn_count_products_by_category(p_category_id INT)
RETURNS INT
DETERMINISTIC
BEGIN
RETURN (
SELECT COUNT(*)
FROM products
WHERE category_id = p_category_id
AND deleted = 0
);
END$$

DELIMITER ;
```

Mục đích: Đếm số sản phẩm trong một category (chưa bị xóa mềm).

3.5 Order

3.5.1 Table orders

3.5.1.1 Cấu trúc bảng

Bảng `orders` lưu trữ thông tin các đơn hàng.

```
CREATE TABLE `orders` (
    `id` varchar(255) NOT NULL,
    `created_at` datetime(6) NOT NULL,
    `deleted` tinyint(1) DEFAULT '0',
    `shipping_district` varchar(255) DEFAULT NULL,
```

```
    `product_name` varchar(255) NOT NULL,
    `shipping_province` varchar(255) DEFAULT NULL,
    `quantity` int NOT NULL,
    `shipping_phone_number` varchar(255) NOT NULL,
    `status` enum('PENDING','ACCEPTED','CANCELLED_BY_CLIENT','CANCELLED_BY_SH
    `thumbnail` varchar(255) NOT NULL,
    `total_price` decimal(15,2) NOT NULL,
    `updated_at` datetime(6) DEFAULT NULL,
    `variant_name` varchar(255) DEFAULT NULL,
    `shipping_ward` varchar(255) DEFAULT NULL,
    `client_id` varchar(255) NOT NULL,
    `seller_id` varchar(255) NOT NULL,
    `variant_id` varchar(255) NOT NULL,
    PRIMARY KEY (`id`),
    KEY `idx_orders_status` (`status`),
    KEY `FKojjigrbyd7qrcwrxvr7e9bdr2` (`client_id`),
    KEY `FKsb9w6305d2be0rwbtifi7wymp` (`seller_id`),
    KEY `FKia12q7m9rrpxq1nku1ea4o9lr` (`variant_id`),
    CONSTRAINT `FKia12q7m9rrpxq1nku1ea4o9lr` FOREIGN KEY (`variant_id`) REFER
    CONSTRAINT `FKojjigrbyd7qrcwrxvr7e9bdr2` FOREIGN KEY (`client_id`) REFERE
    CONSTRAINT `FKsb9w6305d2be0rwbtifi7wymp` FOREIGN KEY (`seller_id`) REFERE
);
```

3.5.1.2 Index

```
    CREATE INDEX idx_fk_client_id ON orders(client_id);
    CREATE INDEX idx_fk_seller_id ON orders(seller_id);
    CREATE INDEX idx_fk_variant_id ON orders(variant_id);
    CREATE INDEX idx_client_id_status ON orders(client_id, status);
```

3.5.1.3 View

```
    CREATE VIEW order_client_seller_view AS
 SELECT id, created_at, thumbnail,product_name, variant_name, quantity, total
```

```
FROM orders
   WHERE deleted = false;
```

3.5.1.4 Trigger

```
CREATE TRIGGER trg_orders_updated_at
BEFORE UPDATE ON orders
FOR EACH ROW
BEGIN
    SET NEW.updated_at = NOW();
END //

CREATE TRIGGER trg_orders_soft_delete
BEFORE DELETE ON orders
FOR EACH ROW
BEGIN
SET OLD.deleted = true;
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Use soft delete instead';
END //

CREATE TRIGGER trg_orders_order_tracking_location_insert
AFTER UPDATE ON orders
FOR EACH ROW
BEGIN
    IF OLD.status = 'PENDING' AND NEW.status = 'ACCEPTED' THEN
        INSERT INTO order_tracking_locations(id, address, arrived_at,
        create_at, deleted, remaining_distance, updated_at, order_id)
        VALUES (UUID(), CONCAT(shipping_ward, ',',shipping_district,
        ',' ,shipping_province), NOW(), NOW(), false, 0.000, NOW(), id)
    END IF;
END //

CREATE TRIGGER trg_orders_update_log
AFTER UPDATE ON orders
```

```
FOR EACH ROW
BEGIN
    INSERT INTO orders_update_log(id, order_id, updated_at)
    VALUES (UUID(), id, NOW());
END //

CREATE TRIGGER trg_orders_delete_log
AFTER UPDATE ON orders
FOR EACH ROW
BEGIN
    IF OLD.deleted != NEW.deleted THEN
        INSERT INTO orders_delete_log(id, order_id,
        updated_at) VALUES (UUID(), id, NOW())
    END IF;
END //
```

3.5.1.5 Procedure

```
CREATE PROCEDURE get_orders_by_client(
    IN p_client_id VARCHAR(255)
)
BEGIN
    SELECT *
    FROM order_client_seller_view
    WHERE client_id = p_client_id;
END //

CREATE PROCEDURE get_orders_by_client(
    IN p_seller_id VARCHAR(255)
)
BEGIN
    SELECT *
    FROM order_client_seller_view
    WHERE seller_id = p_seller_id;
END //
```

```
CREATE PROCEDURE get_orders_by_client_status(
    IN p_client_id VARCHAR(255),
    IN p_status VARCHAR(255)
)
BEGIN
    SELECT *
    FROM order_client_seller_view
    WHERE client_id = p_client_id AND
            status = p_status;
END //

CREATE PROCEDURE get_orders_by_seller_status(
    IN p_seller_id VARCHAR(255),
    IN p_status VARCHAR(255)
)
BEGIN
    SELECT *
    FROM order_client_seller_view
    WHERE seller_id = p_seller_id AND
            status = p_status;
END //
```

3.5.1.6 Function

```
CREATE FUNCTION count_orders_by_client(
    p_client_id VARCHAR(255)
)
RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE result INT;
    SELECT COUNT(*) INTO result
    FROM order_client_seller_view
    WHERE client_id = p_client_id;
```

```
        RETURN results;
    END //

    CREATE FUNCTION count_orders_by_seller(
        p_seller_id VARCHAR(255)
    )
    RETURNS INT
    DETERMINISTIC
    BEGIN
        DECLARE result INT;
        SELECT COUNT(*) INTO result
        FROM order_client_seller_view
        WHERE seller_id = p_seller_id;
        RETURN results;
    END //
```

3.5.2 Table order_tracking_locations

3.5.2.1 Cấu trúc bảng

Bảng `order_tracking_locations` lưu thông tin theo dõi vị trí đơn hàng.

```
CREATE TABLE `order_tracking_locations` (
    `id` varchar(255) NOT NULL,
    `address` varchar(255) NOT NULL,
    `arrived_at` datetime(6) NOT NULL,
    `created_at` datetime(6) NOT NULL,
    `deleted` tinyint(1) DEFAULT '0',
    `remaining_distance` float NOT NULL,
    `updated_at` datetime(6) DEFAULT NULL,
    `order_id` varchar(255) NOT NULL,
    PRIMARY KEY (`id`),
    KEY `idx_order_id` (`order_id`),
    CONSTRAINT `FK3wof50lvhiqf1kwe3427our99` FOREIGN KEY (`order_id`) REFEREN
```

```
);
```

3.5.2.2 Index

```
CREATE INDEX idx_fk_order_id ON order_tracking_locations(order_id);
```

3.5.2.3 View

```
CREATE VIEW order_tracking_detail_view AS
SELECT o.id, o.created_at, thumbnail ,product_name, variant_name, quantity,
FROM orders o
   JOIN order_tracking_locations otl on o.id = otl.order_id
   WHERE o.deleted = false and otl.deleted = false ;
```

3.5.2.4 Trigger

```
CREATE TRIGGER trg_order_tracking_locations_update_log
AFTER UPDATE ON order_tracking_locations
FOR EACH ROW
BEGIN
    INSERT INTO order_tracking_locations_update_log(id, order_id, updatec
    VALUES (UUID(), id, NOW());
END;

CREATE TRIGGER trg_order_tracking_locations_delete_log
AFTER UPDATE ON orders
FOR EACH ROW
BEGIN
    IF OLD.deleted != NEW.deleted THEN
        INSERT INTO order_tracking_locations_delete_log(id, order_id,
        updated_at) VALUES (UUID(), id, NOW())
    END IF;
END;
```

3.5.2.5 Procedure

3.5.2.6 Function

3.6 Transaction

Bảng `transactions` lưu trữ thông tin giao dịch thanh toán.

3.6.1 Cấu trúc bảng

3.6.2 Index

3.6.3 View

3.6.4 Trigger

3.6.4.1 Procedure

3.6.4.2 Function

3.7 Notification

3.7.1 Cấu trúc bảng

```
CREATE TABLE notifications (
    id          VARCHAR(255)  NOT NULL,
    title       VARCHAR(255)  NOT NULL,
    content     VARCHAR(255)  NOT NULL,
    type        ENUM('ORDER','TRANSACTION','PRODUCT',
                     'ACCOUNT','SYSTEM')          DEFAULT NULL,
    image       VARCHAR(255)                      DEFAULT NULL,
    target_id   VARCHAR(255)                      DEFAULT NULL,
    is_read     TINYINT(1)    NOT NULL             DEFAULT 0,
    read_at     DATETIME(6)                       DEFAULT NULL,
    created_at  DATETIME(6)   NOT NULL,
```

```
    updated_at  DATETIME(6)                          DEFAULT NULL,
    deleted     TINYINT(1)                           DEFAULT 0,
    user_id     VARCHAR(255)  NOT NULL,
    PRIMARY KEY (id),
    FOREIGN KEY (user_id) REFERENCES users(id)
);
```

3.7.2 Index

```
CREATE INDEX idx_user ON notifications(user_id);
```

3.7.3 View

user_notifications_view — view hiển thị các thông báo chưa đọc.

```
CREATE VIEW user_notifications_view AS
SELECT
    id,
    user_id,
    title,
    type,
    content,
    image,
    target_id,
    is_read,
    read_at,
    created_at
FROM notifications
WHERE deleted = 0
ORDER BY created_at DESC;
```

notification_unread_badge_view — đếm số thông báo chưa đọc theo từng người dùng.

```
CREATE VIEW notification_unread_badge_view AS
```

```
SELECT
    user_id,
    COUNT(id) AS unread_count
FROM notifications
WHERE deleted = 0
  AND is_read = 0
GROUP BY user_id;
```

3.7.4 Trigger

trg_notifications_before_insert — kiểm tra và set id, set created_at, khởi tạo is_read = 0 và read_at = NULL.

```
DELIMITER $$
CREATE TRIGGER trg_notifications_before_insert
BEFORE INSERT ON notifications
FOR EACH ROW
BEGIN
    IF NEW.id IS NULL OR NEW.id = '' THEN
        SET NEW.id = UUID();
    END IF;
    IF NEW.created_at IS NULL THEN
        SET NEW.created_at = NOW(6);
    END IF;
    SET NEW.is_read = 0;
    SET NEW.read_at  = NULL;
END$$
DELIMITER ;
```

trg_notifications_before_update — cập nhật read_at khi is_read chuyển từ 0 sang 1.

```
DELIMITER $$
CREATE TRIGGER trg_notifications_before_update
BEFORE UPDATE ON notifications
FOR EACH ROW
```

```
BEGIN
    IF OLD.is_read = 0 AND NEW.is_read = 1 THEN
        SET NEW.read_at = NOW(6);
    END IF;
END$$
DELIMITER ;
```

3.8 Shopping Cart

3.8.1 Table shopping_carts

3.8.1.1 Cấu trúc bảng

```
CREATE TABLE shopping_carts (
    id             VARCHAR(255) NOT NULL,
    total_quantity INT          NOT NULL,
    created_at     DATETIME(6)  DEFAULT NULL,
    updated_at     DATETIME(6)  DEFAULT NULL,
    deleted        TINYINT(1)   DEFAULT 0,
    client_id      VARCHAR(255) NOT NULL,
    PRIMARY KEY (id),
    FOREIGN KEY (client_id) REFERENCES users(id)
);
```

3.8.1.2 Index

```
CREATE INDEX idx_shopping_carts_client
    ON shopping_carts(client_id);
```

3.8.1.3 Trigger

```
DELIMITER $$
CREATE TRIGGER trg_shopping_carts_before_insert
BEFORE INSERT ON shopping_carts
FOR EACH ROW
```

```
BEGIN
    IF NEW.id IS NULL OR NEW.id = '' THEN
        SET NEW.id = UUID();
    END IF;
    IF NEW.created_at IS NULL THEN
        SET NEW.created_at = NOW(6);
    END IF;
    SET NEW.total_quantity = 0;
END$$
DELIMITER ;
```

3.8.2 Table shopping_cart_items

3.8.2.1 Cấu trúc bảng

```
CREATE TABLE shopping_cart_items (
    id               VARCHAR(255)  NOT NULL,
    price_each       DECIMAL(15,2) NOT NULL,
    quantity         INT           NOT NULL,
    created_at       DATETIME(6)   DEFAULT NULL,
    updated_at       DATETIME(6)   DEFAULT NULL,
    deleted          TINYINT(1)    DEFAULT 0,
    shopping_cart_id VARCHAR(255)  NOT NULL,
    variant_id       VARCHAR(255)  NOT NULL,
    PRIMARY KEY (id),
    FOREIGN KEY (shopping_cart_id) REFERENCES shopping_carts(id),
    FOREIGN KEY (variant_id)       REFERENCES variants(id)
);
```

3.8.2.2 Index

```
CREATE INDEX idx_shopping_cart_items_cart
    ON shopping_cart_items(shopping_cart_id);
```

3.8.2.3 Trigger

```
DELIMITER $$
CREATE TRIGGER trg_shopping_cart_items_before_insert
BEFORE INSERT ON shopping_cart_items
FOR EACH ROW
BEGIN
    IF NEW.id IS NULL OR NEW.id = '' THEN
        SET NEW.id = UUID();
    END IF;
    IF NEW.created_at IS NULL THEN
        SET NEW.created_at = NOW(6);
    END IF;
END$$
DELIMITER ;
```

3.8.3 View

cart_details_view — View hiển thị chi tiết sản phẩm có trong giỏ hàng.

```
CREATE VIEW cart_details_view AS
SELECT
    sci.id                          AS item_id,
    sc.client_id,
    sci.variant_id,
    sci.quantity,
    sci.price_each,
    (sci.quantity * sci.price_each) AS item_total_price,
    v.thumbnail,
    p.name                          AS product_name
FROM       shopping_cart_items sci
JOIN       shopping_carts sc ON sci.shopping_cart_id = sc.id
JOIN       variants        v  ON sci.variant_id       = v.id
JOIN       products        p  ON v.product_id         = p.id
WHERE      sci.deleted = 0
  AND      sc.deleted  = 0;
```

3.9 Refund

3.9.1 Table order_refund_requests

Bảng `order_refund_requests` lưu trữ thông tin các yêu cầu hoàn hàng

3.9.1.1 Cấu trúc bảng

```
CREATE TABLE `order_refund_requests` (
`id` varchar(255) NOT NULL,
`created_at` datetime(6) NOT NULL,
`deleted` tinyint(1) DEFAULT '0',
`reason` text NOT NULL,
`refund_amount` decimal(38,2) NOT NULL,
`refund_type` enum('CLIENT_REFUND','SHIPPER_REFUND') NOT NULL,
`resolved_at` datetime(6) DEFAULT NULL,
`status` enum('PENDING','REJECTED','APPROVED') NOT NULL,
`updated_at` datetime(6) DEFAULT NULL,
`order_id` varchar(255) DEFAULT NULL,
`requester_id` varchar(255) DEFAULT NULL,
`seller_id` varchar(255) DEFAULT NULL,
PRIMARY KEY (`id`),
KEY `idx_order_refund_requests_seller` (`seller_id`),
KEY `FKi7t1qg2r9e076896v9l8tg48t` (`order_id`),
KEY `FK9h3lcnhbj827axa4ko08wepwp` (`requester_id`),
CONSTRAINT `FK9h3lcnhbj827axa4ko08wepwp` FOREIGN KEY (`requester_id`)
REFERENCES `users` (`id`),
CONSTRAINT `FKdn4av77nioo6h57f7h4k8jypm` FOREIGN KEY (`seller_id`)
REFERENCES `users` (`id`),
CONSTRAINT `FKi7t1qg2r9e076896v9l8tg48t` FOREIGN KEY (`order_id`)
REFERENCES `orders` (`id`)
);
```

3.9.1.2 Index

```
CREATE INDEX idx_fk_order_id ON order_refund_requests(order_id);
CREATE INDEX idx_fk_requester_id ON order_refund_requests(requester_id);
CREATE INDEX idx_requester_id_status ON order_refund_requests(requester_i
CREATE INDEX idx_order_id_status ON order_refund_requests(order_id, statu
```

3.9.1.3 View

```
   CREATE VIEW order_refund_request_client_seller_view AS
SELECT id, created_at, reason, refund_amount, refund_type,
   resolved_at, status, updated_at,order_id,
   requester_id, seller_id
FROM order_refund_requests
WHERE deleted = false;
```

3.9.1.4 Trigger

```
CREATE TRIGGER trg_orders_update_log
AFTER UPDATE ON orders
FOR EACH ROW
BEGIN
    INSERT INTO order_refund_request_update_log(id,
    order_refund_request_id, updated_at)
    VALUES (UUID(), id, NOW());
END //

CREATE TRIGGER trg_orders_delete_log
AFTER UPDATE ON orders
FOR EACH ROW
BEGIN
    IF OLD.deleted != NEW.deleted THEN
        INSERT INTO orders_delete_log(id,
        order_refund_request_id, updated_at)
        VALUES (UUID(), id, NOW())
```

```
    END IF;
END //

CREATE TRIGGER trg_orders_updated_at
BEFORE UPDATE ON order_refund_requests
FOR EACH ROW
BEGIN
    SET NEW.updated_at = NOW();
END //
```

3.9.1.5 Procedure

```
CREATE PROCEDURE GetRefundRequestByClient(IN p_client_id INT)
BEGIN
    SELECT * from order_refund_request
    WHERE requester_id = p_client_id AND deleted = false;
END //

CREATE PROCEDURE GetRefundRequestByProduct(IN p_product_id INT)
BEGIN
    SELECT ofr.* from order_refund_request ofr
    JOIN orders o on o.id = ofr.order_id
    JOIN variants v on v.id = o.variant_id
    JOIN products p on p.id = v.product_id
    WHERE p.id = p_product_id AND and ofr.deleted = false;
END //

CREATE PROCEDURE GetRefundRequestByClient(IN p_seller_id INT)
BEGIN
    SELECT * from order_refund_request
    WHERE seller = p_seller_id AND deleted = false;
END //

CREATE PROCEDURE GetRefundRequestByResolvedAt(
```

```
    IN p_below_date date,
    IN p_above_date date
)
BEGIN
    SELECT * from order_refund_request
    WHERE resolved_at >= p_below_date AND
     resolved_at <= p_above_date;
END //
```

3.9.1.6 Function

```
CREATE FUNCTION count_refund_request_by_seller(
    p_seller_id VARCHAR(255)
)
RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE result INT;
    SELECT COUNT(*) INTO result
    FROM order_refund_request_client_seller_view
    WHERE seller_id = p_seller_id;
    RETURN results;
END //

CREATE FUNCTION count_refund_request_by_client(
    p_client_id VARCHAR(255)
)
RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE result INT;
    SELECT COUNT(*) INTO result
    FROM order_refund_request_client_seller_view
    WHERE client_id = p_client_id;
```

```
    RETURN results;
END //
```

Chức năng là linh hồn của hệ thống.
Phát triển phần mềm

Chapter 4

Triển khai các chức năng bổ sung cho hệ thống

Các chức năng bổ sung đóng vai trò quan trọng trong việc nâng cao trải nghiệm người dùng và hỗ trợ quyết định kinh doanh. Phần này trình bày chi tiết cài đặt từng tính năng bao gồm giải thích cơ chế, cài đặt vật lý và câu lệnh truy vấn.

4.1 Chức năng cho guest (chưa đăng nhập)

4.1.1 Đăng ký

4.1.1.1 Giải thích cơ chế

Chức năng đăng ký cho phép người dùng tạo tài khoản mới. Hệ thống sẽ:

1. Kiểm tra email đã tồn tại chưa
2. Hash mật khẩu trước khi lưu
3. Tạo bản ghi trong bảng `accounts`
4. Gửi email xác nhận

4.1.1.2 Cài đặt vật lý

```
CREATE TABLE registration_log (
    registration_id INT PRIMARY KEY AUTO_INCREMENT,
    email VARCHAR(255),
    registration_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    status ENUM('pending', 'verified', 'failed')
);

CREATE TRIGGER trg_log_new_registration
AFTER INSERT ON accounts
FOR EACH ROW
BEGIN
    INSERT INTO registration_log (email, status)
    VALUES (NEW.email, 'pending');
END;
```

4.1.1.3 Câu lệnh truy vấn

```
-- Kiểm tra email tồn tại
SELECT account_id FROM accounts WHERE email = ?;

-- Thêm tài khoản mới
INSERT INTO accounts (email, password_hash, role, status)
VALUES (?, SHA2(?, 256), 'buyer', 'active');

-- Kiểm tra đăng ký mới nhất
SELECT * FROM accounts
WHERE created_at >= DATE_SUB(NOW(), INTERVAL 24 HOUR)
ORDER BY created_at DESC;
```

4.1.2 Đăng nhập

4.1.2.1 Giải thích cơ chế

4.1.2.2 Cài đặt vật lý

4.1.2.3 Câu lệnh truy vấn

4.1.3 Đăng xuất

4.1.3.1 Giải thích cơ chế

4.1.3.2 Cài đặt vật lý

4.1.3.3 Câu lệnh truy vấn

4.1.4 Hiển thị danh mục được tìm kiếm nhiều

4.1.4.1 Giải thích cơ chế

4.1.4.2 Cài đặt vật lý

4.1.4.3 Câu lệnh truy vấn

4.1.5 Hiển thị sản phẩm bán chạy

4.1.5.1 Giải thích cơ chế

4.1.5.2 Cài đặt vật lý

4.1.5.3 Câu lệnh truy vấn

4.1.6 Hiển thị sản phẩm xem nhiều nhất

4.1.6.1 Giải thích cơ chế

4.1.6.2 Cài đặt vật lý

4.1.6.3 Câu lệnh truy vấn

4.1.7 Lọc sản phẩm đa tiêu chí

4.1.7.1 Giải thích cơ chế

Bibliography

[1] Manfred Eigen. Selforganization of matter and the evolution of biological macromolecules. *Naturwissenschaften*, 58(10):465–523, 1971.

[2] Donald E Knuth. Semantics of context-free languages. *Mathematical Systems Theory*, 2(2):127–145, 1968.

Appendix A

Conclusion

A.1 Điểm mạnh

Hệ thống cơ sở dữ liệu được thiết kế với các điểm mạnh sau:

- Hiệu suất cao và khả năng mở rộng
- Bảo mật dữ liệu toàn diện
- Dễ dàng quản lý và bảo trì

A.2 Điểm yếu

Các hạn chế của hệ thống hiện tại:

- Cần tối ưu hóa thêm cho các truy vấn phức tạp
- Yêu cầu quản trị viên chuyên biệt

A.3 Kết luận Cuối

Hệ thống cơ sở dữ liệu này cung cấp một nền tảng vững chắc cho ứng dụng e-commerce, hỗ trợ các chức năng kinh doanh cơ bản và nâng cao. Với sự hoàn thiện thêm, nó sẽ trở thành một giải pháp toàn diện cho quản lý dữ liệu e-commerce.

Appendix B

Khuyến nghị và Hướng phát triển

Các khuyến nghị cho phát triển tương lai của hệ thống:

- Tích hợp công nghệ machine learning để phân tích dữ liệu khách hàng
- Cải thiện hiệu suất với caching layer
- Triển khai các giải pháp backup và disaster recovery
- Mở rộng hỗ trợ cho các thị trường quốc tế